



School of Computer Science and Artificial Intelligence

Lab Assignment # 07

Course Title : **AI Assistant Coding**

Name of Student : **M. Sai Pallavi**

Enrollment No. : **2303A54063**

Batch No. : **48**

Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., print "Hello"). Use AI to detect and fix the syntax error.

Prompt: “fix the syntax error caused by missing parentheses in the print statement. Explain why the error occurs and verify the corrected code using at least 3 assert test cases.”

Requirements:

- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code

Works.

Expected Output #1:

- Corrected code with proper syntax and AI explanation.

Output Screenshot:

```
syntax_error.py > ...
1  #Provide a Python snippet with a missing parenthesis in a printstatement like print "Hello".
2  def greet():
3  |   print("Hello, AI Debugging Lab!")
4  greet()
5  # The syntax error occurs because the print statement is not using parentheses, which is requ
6  def greet():
7  |   print("Hello, AI Debugging Lab!")
8  # Test cases to verify the corrected function works properly
9  assert greet() is None # The function should return None since it only prints a message
10 # Additional test cases to ensure the function behaves as expected
11 import io
12 import sys
13 # Capture the output of the greet function
14 captured_output = io.StringIO()
15 sys.stdout = captured_output # Redirect stdout to capture the print output
16 greet() # Call the function to capture its output
17 sys.stdout = sys.__stdout__ # Reset redirect.
18 assert captured_output.getvalue() == "Hello, AI Debugging Lab!\n" # Check if the output is c
19 # Test case to ensure that the function does not return any value
20 assert greet() is None # The function should return None since it only prints a message
21 print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/syntax_error.py"
Hello, AI Debugging Lab!
Hello, AI Debugging Lab!
Hello, AI Debugging Lab!
All test cases passed!
```

Task Description #2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Prompt: “Fix the bug caused by using assignment (=) instead of comparison (==) in the if condition. Explain why it is incorrect and validate the fix with 3 assert test cases.”

Expected Output #2:

- Corrected code using == with explanation and successful test execution

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Output Screenshot:

```
if_condition.py > ...
1  #correct the code and verify the corrected version using 3 assert test cases.Find and explain
2  def check_number(n):
3      if n == 10:
4          return "Ten"
5      else:
6          return "Not Ten"
7  # The error in the original code is that it uses a single equals sign (=) instead of a double
8  # Test cases to verify the corrected function works properly
9  assert check_number(10) == "Ten" # Test case where n is equal to 10
10 assert check_number(5) == "Not Ten" # Test case where n is not equal to 10
11 assert check_number(0) == "Not Ten" # Test case where n is not equal to 10
12 print("All test cases passed!")
13 # In the corrected code, we use the double equals sign (==) to compare the value of n with 10
14
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/syntax_error.py"
All test cases passed!
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/if_condition.py"
```

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Prompt: “Fix the runtime error when a file is missing by applying try-except error handling. Display user-friendly messages and test the code for existing and missing files.”

Requirements:

- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output #3:

- Safe file handling with exception management.

Output Screenshot:

```
file_not_found.py > ...
1  #give the Python code so it does not crash when a file is missing. Add try-except error handling
2  def read_file(filename):
3      with open(filename, 'r') as f:
4          return f.read()
5  # Test cases to verify the function handles missing files properly
6  try:
7      content = read_file("existing_file.txt") # Assuming this file exists
8      print("File content:", content)
9  except FileNotFoundError:
10     print("Error: The file 'existing_file.txt' was not found.")
11  try:
12     content = read_file("missing_file.txt") # This file does not exist
13     print("File content:", content)
14  except FileNotFoundError:
15     print("Error: The file 'missing_file.txt' was not found.")
16  try:
17     content = read_file("another_missing_file.txt") # This file also does not exist
18     print("File content:", content)
19  except FileNotFoundError:
20     print("Error: The file 'another_missing_file.txt' was not found.")
21  # The try-except block is used to catch the FileNotFoundError that occurs when trying to open
22
```

314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/if_condition.py"

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/file_not_found.py"

File content: This is a test file.

Error: The file 'missing_file.txt' was not found.

Error: The file 'another_missing_file.txt' was not found.

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>

Task Description #4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g.,obj.undefined_method()). Use AI to debug and fix.

Prompt: “Fix the Car class bug where drive() is called but not defined, and validate the corrected class with 3 assert tests.”

Requirements:

- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

- Corrected class with clear AI explanation.

Output Screenshot:

```
undefined_method.py > ...
1  #give the python code verify the corrected class using 3 assert test cases.The following Pyth
2  class Car:
3      def start(self):
4          return "Car started"
5      def drive(self):
6          return "Car is driving"
7  my_car = Car()
8  print(my_car.drive()) # drive() is not defined
9  # The error occurs because the Car class does not have a method named drive(). When we try to
10 class Car:
11     def start(self):
12         return "Car started"
13
14     def drive(self):
15         return "Car is driving"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/file_not_found.py"
Error: The file 'another_missing_file.txt' was not found.
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/undefined_method.py"
Car is driving
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>
```

Task Description #5 (TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Prompt: “Fix the TypeError caused by adding a string and an integer. Provide two solutions: type casting and string concatenation, and validate both using 3 assert test cases.”

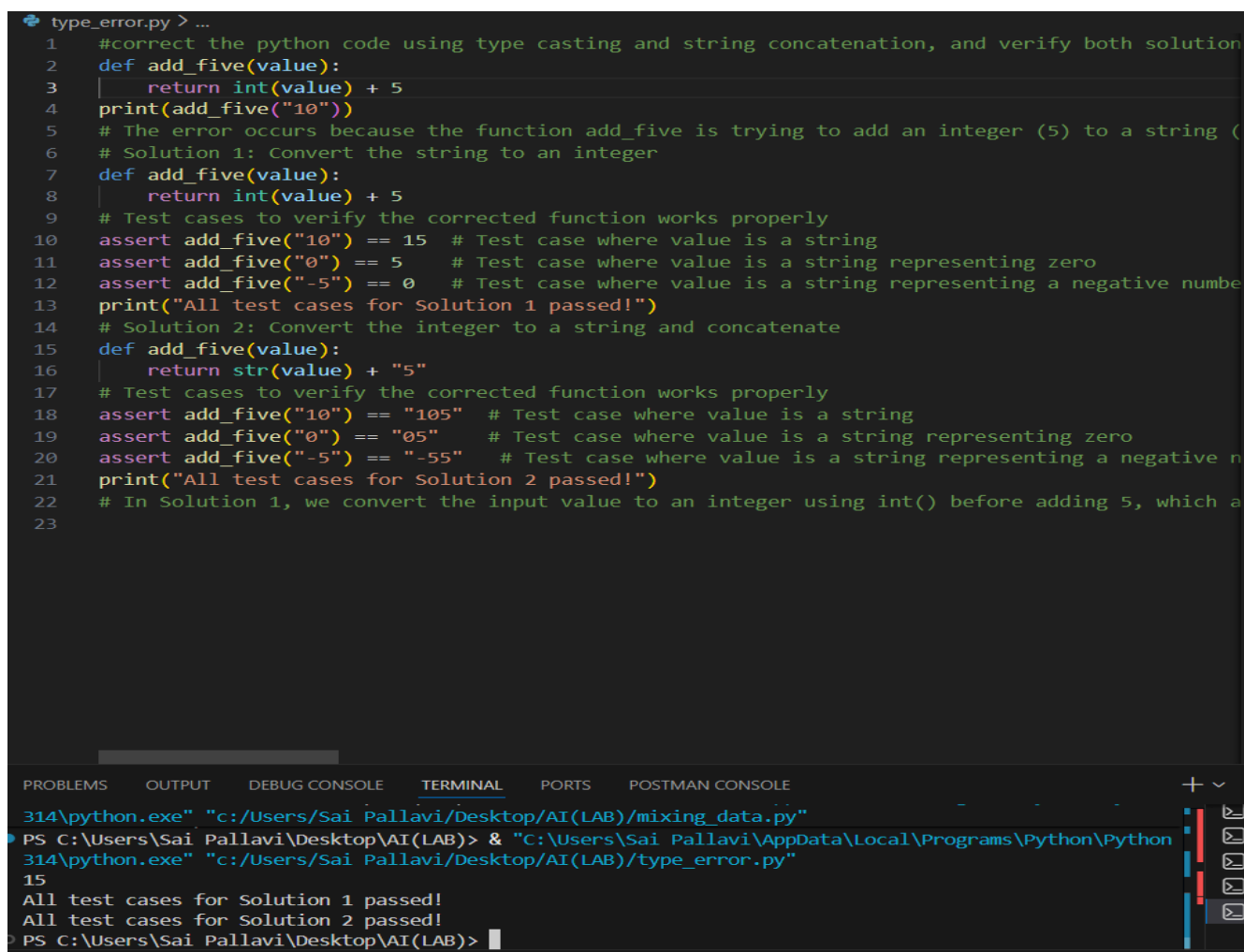
Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output #5:

- Corrected code that runs successfully for multiple inputs.

Output Screenshot:



```
type_error.py > ...
1  #correct the python code using type casting and string concatenation, and verify both solutions
2  def add_five(value):
3      |   return int(value) + 5
4  print(add_five("10"))
5  # The error occurs because the function add_five is trying to add an integer (5) to a string (
6  # Solution 1: Convert the string to an integer
7  def add_five(value):
8      |   return int(value) + 5
9  # Test cases to verify the corrected function works properly
10 assert add_five("10") == 15 # Test case where value is a string
11 assert add_five("0") == 5   # Test case where value is a string representing zero
12 assert add_five("-5") == 0  # Test case where value is a string representing a negative number
13 print("All test cases for Solution 1 passed!")
14 # Solution 2: Convert the integer to a string and concatenate
15 def add_five(value):
16     |   return str(value) + "5"
17 # Test cases to verify the corrected function works properly
18 assert add_five("10") == "105" # Test case where value is a string
19 assert add_five("0") == "05"   # Test case where value is a string representing zero
20 assert add_five("-5") == "-55" # Test case where value is a string representing a negative number
21 print("All test cases for Solution 2 passed!")
22 # In Solution 1, we convert the input value to an integer using int() before adding 5, which a
23
```

314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/mixing_data.py"

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python

314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/type_error.py"

15

All test cases for Solution 1 passed!

All test cases for Solution 2 passed!

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>